What If AI Didn't Need to Read or Write Code?

Rethinking programming languages, reusable software, and the future of AI-driven development.

It started with R2-D2

I've spent more than 25 years developing software, but this particular idea did not begin with a programming language or a compiler. It began with R2-D2.

R2-D2 doesn't speak to us in ordinary words. He beeps and boops. Yet inside the story, other systems understand his signals. A compact signal can stand for information, an instruction, or an entire familiar operation. The important thing is not the sound; it is that both sides agree on what the signal means.

That got me thinking about AI coding agents. We have trained them to produce `public`, `void`, `for`, `if`, and the rest of the languages humans developed so programmers could communicate with machines. But if the author and the consumer are increasingly machines, do they really need those words? Could an AI use a more direct vocabulary of operations—and, more importantly, a short reference to a complex function it already knows exists?

R2-D2 is an analogy, not evidence about how real robots encode messages. But it leads to two serious questions:

Why are AI agents still expected to write human-oriented source code? And why should they keep reading or recreating functionality that already exists?

The second question may be more valuable than the first. The experiments described later in this article do not settle either question, but they reveal what such a system would have to prove.

The hidden inefficiency in AI-generated software

Consider a common development request:

“Create a service that monitors inventory, tracks changes, generates alerts, and synchronizes data between locations.”

An AI agent can probably write much of this application.

But what actually happens?

The agent examines existing source code, searches for libraries, generates functions, builds data structures, implements business logic, and tests the resulting software.

Some of that work is genuinely new.

Much of it, however, involves functionality implemented countless times before.

Data validation, scheduling, synchronization, logging, authentication, transaction management, and numerous other operations appear repeatedly across different applications.

Modern software developers already use libraries to avoid reinventing these functions.

But AI agents still spend substantial computational resources discovering, interpreting, adapting, and sometimes recreating functionality they could theoretically reuse directly.

The cost is not limited to generated code.

Every time an AI agent processes unnecessary source code, it consumes context, computational resources, and potentially additional reasoning and verification effort.

What if the agent didn't need to read that code at all?

A shared repository of executable functionality

Imagine an environment where AI agents have access to a shared repository containing previously developed, validated software capabilities.

Each capability has a unique identity, defined inputs and outputs, known behavior, dependencies, permissions, and an executable implementation.

Instead of asking an AI agent to recreate inventory synchronization logic, the agent could discover an existing capability that already performs the required function.

Once compatibility has been established, using that functionality might require only a compact reference:

```
!4821(s,d,p)
```

This is a **hypothetical machine-facing reference**, not a final syntax decision. In a future system, even these visible characters might be replaced by dedicated operation tokens. The values `s`, `d`, and `p` represent inputs and policy; their meanings and permitted uses would be established by a versioned binding, not guessed from their names.

The agent would not necessarily need to read the implementation, generate another version, or even repeatedly inspect its full specification.

A trusted execution layer would resolve the reference, validate its authorization and binding, and execute the existing implementation.

The distinction is important.

The AI is not copying code into another application.

It is composing software from capabilities whose behavior has already been established for a particular set of assumptions.

Now imagine the repository contains thousands—or eventually millions—of reusable capabilities.

An agent could assemble substantial portions of a new application by selecting, connecting, and configuring existing functionality.

Only genuinely new behavior would require new implementation.

This could transform development from a process dominated by generating source code into one increasingly focused on **composing established computational functionality**.

There are important limitations: two functions that appear similar may behave differently, and compatibility must be verified rather than assumed. But the potential efficiency improvement deserves investigation.

Our recommended research architecture: The AI Computational Commons

After testing and debating this idea with several AI systems, I would recommend that researchers investigate an **AI Computational Commons**: a governed ecosystem of shared, executable capabilities that multiple AI agents and providers can discover, verify, and compose without repeatedly reading the underlying implementations.

A public commons would contain common functionality such as database access, file and stream processing, communication protocols, image transforms, queries, data structures, algorithms, and design patterns. Agents could agree on what each capability guarantees and which variants exist; they need not agree that one implementation is best for every environment. Fast, memory-efficient, cloud-native, and restricted-execution variants might satisfy the same interface under different conditions.

An application would also have **private capability domains**, protecting proprietary business logic and confidential information. Public capabilities could be visible to everyone, while organization-, project-, and agent-scoped implementations would be accessible only to authorized execution

hosts. Encryption, identity checks, and execution permissions are separate protections: a signed agent should not automatically be able to decrypt or execute private code.

The key element would be a **validated binding**. The agent identifies a suitable capability, the host verifies relevant compatibility and authority, and a scoped reference is established. Later calls could use that reference without repeating a full implementation or contract in model context. The host—not the AI's confidence—would recheck revocation, policy, artifact identity, and other conditions that affect the binding.

A successful public commons could make software development increasingly cumulative. Agents would spend attention selecting safe, appropriate building blocks, and create new functionality only for the parts that genuinely differ from established patterns.

This is a strong research recommendation, not a claim that the architecture has been proven to reduce cost or is ready to replace existing software infrastructure. Contemporary package managers and coding agents already reuse functions without reading their source, so any proposed commons must outperform that strong baseline rather than an unrealistic all-code-regenerated scenario. The percentage of any application that can be assembled from shared capabilities remains an open measurement, not a fixed 60–80% result.

Do we need human programming languages in the middle?

The shared repository addresses one part of the problem.

The next question is what happens when an AI genuinely needs to develop new functionality.

Consider this simple C# example:

```
public void Main(int NumOfRuns)
{
    for (int i = 0; i < NumOfRuns; i++)
    {
        if (i == 0)
        {
            m_log.Info(
                $"Starting controller service for {NumOfRuns} runs");
        }
        else
        {
            m_log.Info(
                $"Restarting controller service for run {i+1} of {NumOfRuns}");
        }
    }
}
```

Much of the syntax exists to describe operations in a form familiar to human developers.

But an AI agent does not inherently need the words `public`, `void`, `for`, or `else` to represent their computational behavior.

Imagine an alternative symbolic representation:

```
@1(n:I){
  R(n,i){
    ?(i=0):!7(T0,n)
          :!7(T1,i+1,n)
  }
}
```

Here, a predefined vocabulary describes a function, a counted loop, a conditional operation, and an invocation of a local logging primitive. The message templates are stored separately. In this example, `!7` is **not necessarily a shared AICC capability**; a compact invocation syntax could address local primitives or bound public and private capabilities, with the binding environment determining which.

The symbols themselves are not important. What matters is that their computational meaning is precisely defined.

This is **experimental syntax, not a selected design, functioning language specification, or proposed universal opcode table**. The symbols, type identifiers, IDs, punctuation, and control-flow forms could

all change. What we are proposing to investigate is a language whose operations have precise computational semantics without requiring human-language keywords.

A future AI-native language might go significantly further, using dedicated operation identifiers or compact binary instructions rather than textual symbols.

The AI could construct valid computational operations directly. A deterministic compiler or execution engine would interpret those operations according to established rules.

Humans could still view the resulting behavior as C#, Python, or a visual representation, subject to the semantic limitations of each language.

The human programming language would become a way to inspect and interact with the software, rather than necessarily being its original representation.

Of course, shorter syntax does not automatically mean lower AI costs. Existing language models may tokenize unfamiliar symbols inefficiently or make more mistakes when generating them.

That is part of what needs to be researched.

The larger question is whether an AI system designed and trained specifically for such a representation could perform better than one repeatedly generating traditional source code.

Programming without AICC—and programming with it

The two ideas should not be confused. **The proposed AI-native language can express genuinely new algorithms without AICC.** AICC is a separate, optional source of already-established functionality that the language can invoke when reuse is appropriate.

The examples below deliberately alternate between conventional human programming languages and speculative AI-native notation. They are *illustrations for discussion*, not working implementations, benchmark results, finalized grammar, or evidence of reduced model-token cost. The English explanations and capability tables are for human readers; they are

not words the AI would need to generate or interpret at every call. An actual implementation could use dedicated operation tokens, typed operands, and a host-managed capability-binding table.

Example 1: New algorithm in Python—no AICC

Suppose the agent needs to invent a small transformation. There is no reason to publish every unique decision as a reusable capability.

```
def transform(values):
    result = []
    for value in values:
        if value > 100:
            result.append(value * 2)
        else:
            result.append(value + 10)
    return result
```

One *possible* AI-native representation is:

```
@2(a:4){
  b=[]
  R(#a,i){
    x=a[i]
    ?(x>100):b<<(x*2)
            :b<<(x+10)
  }
  ^b
}
```

For human orientation only, `@2` denotes a function, `R` a counted repetition, `?` a branch, `#a` the number of elements, `<<` an append operation, and `^` a return. In this hypothetical example these are **native language operations, not AICC lookups**. The agent authored the new transformation itself. No claim is made that this notation uses fewer tokens in an existing model.

Example 2: Resource lifetime in C++—no AICC

Some problems deserve *built-in language or runtime guarantees*, not library references. For example, memory ownership and release should not depend on an AI remembering every cleanup path.

```
#include <cstdint>
#include <memory>

std::uint8_t preview(std::uint8_t value) {
    auto buffer = std::make_unique<std::uint8_t[]>(65536);
    buffer[0] = value;
    return buffer[0];
} // RAII releases the allocation
```

A speculative AI-native representation might be:

```
@3(v:1){
  %2(65536,5,b){
    b[0]=v
    ^b[0]
  }
}
```

Here, `%2` is a *native, compiler/runtime-enforced scoped-memory operation*. Policy identifier `5` might specify exclusive ownership and release on every exit path. Those guarantees would have to be formally defined and enforced by the execution engine; the notation does not magically provide memory safety. This example uses **no AICC function**.

Example 3: Public AICC capabilities in Java

Now consider operations that already exist and are well established: normalizing records, grouping them into time windows, and producing reports. A conventional Java application might call known libraries:

```
Report createReport(List<Event> events) {
    var normalized = EventLib.normalize(events);
    var groups = EventLib.groupByWindow(normalized, 600);
    return ReportLib.summarize(groups);
}
```

In our *hypothetical* AI-native language, the composition might look like:

```
@4(e:4){
  ^!123(!121(!120(e),600))
}
```

The following table is **human-only documentation**, not part of the machine-facing program:

Reference	Example capability	Visibility
`!120`	Normalize event records	Public AICC

Reference	Example capability	Visibility
`!121`	Group normalized events into defined time windows	Public AICC
`!123`	Produce a structured report	Public AICC

Those numbers are illustrative local bindings, **not global permanent function IDs**. A host would need to resolve them to specified versions, validate input/output contracts and ordering rules, select a compatible implementation, and enforce effects and permissions. The agent would not need to regenerate these implementations merely to compose them.

Example 4: Public and private AICC combined in C

The same application may require proprietary business logic that should never become public. In C#, that might be expressed using a combination of common libraries and a private project component:

```
Report Evaluate(IEnumerable<Event> events)
{
    var normalized = Common.Normalize(events);
    var groups = Common.GroupByWindow(normalized, 600);
    int risk = PrivateRules.CalculateRisk(groups);
    return Common.CreateReport(groups, risk >= 7 ? 2 : 1);
}
```

A possible AI-native composition is:

```
@5(e:4){
  g=!121(!120(e),600)
  s=!901(g)
  ?(s>=7):^!123(g,2)
         :^!123(g,1)
}
```

Here `!120`, `!121`, and `!123` are public capability bindings; `!901` is a **project-private capability** that calculates risk. The comparison `s>=7` is *newly composed logic*, not a library function. Encryption could protect the private artifact at rest and in transit, while a trusted execution host—not an agent signature alone—would control decryption and execution authority. Public capability code can process sensitive data locally; its being public does **not** require sending private data to a public service.

Example 5: Concurrency as a reusable capability, with new application logic

Thread and task management combine common mechanics with application-specific behavior. In C#, a developer can use `Task`, `SemaphoreSlim`, bounded channels, cancellation tokens, and related primitives. AICC could expose a *verified bounded-concurrency pattern* that composes such mechanisms while the application's operation remains private.

One conceptual AI-native expression is:

```
@6(q:4){
  ^!250(q,!902,8,256,7)
}
```

In the human-only description, `!250` is a **public AICC bounded-processing capability**; `!902` is an **authorized private operation** applied to each item. The numeric operands request eight concurrent operations, a queue limit of 256, and a policy referenced by `7`. That policy would have to define cancellation, ordering, backpressure, partial failures, and cleanup exactly. A real implementation must also specify how a private callable can safely be passed to a public capability without exposing secret code or bypassing permissions.

The opportunity here is not that `!250` is shorter than an English function name. It is that a repeatedly needed, difficult concurrency pattern could be selected and executed with previously established guarantees instead of being reimplemented every time.

What these examples do—and do not—demonstrate

The language and AICC are **independent but complementary**. The AI-native language must be capable of original control flow, arithmetic, data structures, and resource management *without* relying on the commons. When suitable functionality already exists, the same language could reference a bound public or private AICC capability instead of regenerating its internals.

A hypothetical native operation token is not just a compressed spelling of an English keyword. It would have its own precisely defined semantics, learned or available to the model, and enforced by an execution system.

The visible characters in these examples are **a research sketch, not a decision about syntax, opcodes, type IDs, compiler architecture, or the final user interface**.

That leads to two separate empirical questions: whether native operation representations make AI authoring more reliable or cost-effective, and whether shared validated capabilities reduce end-to-end development cost compared with ordinary libraries, documentation, and caching. Neither follows automatically from the shortness of the examples.

Software as accumulated computational knowledge

The most interesting possibility emerges when we combine an AI-native language with shared executable functionality.

Suppose an AI agent creates a new operation.

The system verifies its behavior, records its dependencies, identifies its side effects, and stores it as a reusable capability.

Future agents can reference that operation without reconstructing its implementation.

As new applications are developed, the shared repository grows.

Instead of every project beginning with a new sequence of code generation and interpretation, agents can work with an expanding collection of existing computational building blocks.

This introduces a potential distinction between two forms of AI capability.

Generative intelligence is the ability to reason about unfamiliar requirements and create new solutions.

Accumulated computational knowledge is the ability to preserve, discover, and combine previously established solutions.

Today's AI systems already use external tools, retrieval systems, and software libraries. The proposed change is to make accumulated executable functionality a central element of the software-development architecture.

Consider the possibility that an agent already knows which capabilities satisfy a particular requirement.

Why should it repeatedly consume context describing those capabilities?

A validated binding could allow the execution system to retain the detailed contract and compatibility decision while the agent uses a compact reference.

The agent would receive further details when requirements change, a verification assumption becomes invalid, or a problem requires deeper inspection.

In effect, the AI would not need to reconsider everything it had already established.

This doesn't eliminate the need for reasoning. It aims to preserve valid computational decisions so reasoning effort can be directed toward what is new or uncertain.

A different kind of AI-to-AI communication

The same architecture raises another possibility.

If two AI agents operate using a shared computational vocabulary, they may not need to exchange large amounts of human-readable source code.

One agent could communicate a structured instruction, a capability reference, or a composition of previously established operations.

The receiving agent could work with those instructions without needing to reconstruct their meaning from conventional programming-language syntax.

Different agents might even use different local symbolic identifiers while referring to the same underlying computational operations.

Secure communication could use authenticated encryption and authorization protocols, allowing agents to exchange executable references or instructions through controlled channels.

However, changing symbolic identifiers is not equivalent to encryption, and encrypted communication does not guarantee that executable functionality is safe.

The meaningful research opportunity is not secret programming syntax.

It is a reliable, efficient, interoperable protocol through which AI agents can exchange and reuse computational functionality.

What already exists?

Several established technologies provide pieces of this vision.

Compiler intermediate representations, such as LLVM IR and MLIR, demonstrate that software does not need to remain in its original programming language throughout compilation.

WebAssembly provides a portable executable representation and component interfaces that can support software written in different languages.

Traditional package managers already allow software developers to reuse functionality rather than recreate it.

Research systems such as DreamCoder have explored learning reusable programming abstractions from previously solved problems.

Structured editing environments demonstrate that software can be manipulated through representations other than ordinary source-code text.

These are important precedents.

They also mean the underlying ideas are not entirely new.

The research question is whether they can be brought together and extended into a development architecture designed around how AI agents operate.

The distinction would not be merely creating another programming language or package manager.

It would be enabling AI agents to **discover, compose, author, and exchange executable functionality while minimizing unnecessary exposure to underlying implementations**.

What the AI think tank actually found

I did not want this proposal to depend only on my intuition. To challenge it, I organized a series of structured reviews and experiments with **Claude (Anthropic), Gemini (Google), Grok (xAI), and ChatGPT/GPT-6 (OpenAI)**. I asked them to question the idea, execute comparable toy tasks, look for counterexamples, and report what they could actually measure.

This was an **exploratory, AI-assisted research exercise**, not peer-reviewed science. The models were not trained on a new language. They did not perform fully blinded, randomized comparisons across real production projects. Some submitted their code and execution logs, which were later rerun; other measurements remain model-reported. I want to make that distinction clear because it is precisely how we can improve the idea.

Round 7: Can the mechanisms work at all?

The first executable packet compared four paths: ordinary Python API reuse; reuse through an AICC-like capability registry; newly written Python logic; and a small symbolic language interpreted by a toy runtime. All four agents **reported** the following outcomes on the same small test fixtures:

Test	ChatGPT / GPT-6	Claude	Gemini	Grok
Conventional documented API reuse	14/14	14/14	14/14	14/14
Capability-registry reuse	14/14	14/14	14/14	14/14
New Python controller	6/6	6/6	6/6	6/6
Toy symbolic controller	6/6	6/6	6/6	6/6
Elementary binding checks	3/3	3/3	3/3	3/3

What this supports: the sample functions could be invoked through compact references without inspecting their implementations; a limited symbolic program could be parsed and executed; basic registry binding checks were feasible.

What this does not support: that the registry is more efficient than a mature library. Ordinary API reuse passed the same tests *without reading the function bodies*. Nor do successful tests of a toy interpreter establish the performance of an AI model trained to write its own native operations.

Claude supplied particularly useful adversarial evidence. When it attempted to rebuild an event-processing capability from the *written contract alone*, its first implementation passed only **10 of 14 tests**. The description omitted important observable details. After revising its implementation, Claude reported **14/14** and **2,100 matching differential tests**. The lesson is that a capability name, type signature, or reassuring summary is not enough: the behavioral contract must be sufficiently precise to justify reuse.

A separate comparison tested the proposed language's *compactness*. Claude counted **29 model-tokenizer tokens** for a tiny symbolic controller, versus **27 tokens** for a comparably minimal Python version using the same predefined message templates. Among **246 deliberately mutated symbolic programs**, **22** executed with incorrect behavior. These were mutation experiments, **not the model's actual code-generation error rate**. They challenge the assumption that fewer printed characters automatically imply fewer AI tokens or safer programs.

Round 8: Does AICC improve application composition?

The next packet introduced **20 public capabilities**, eight application-composition tasks evaluated with **20 official workflow assertions**, deliberately misleading variants, and simulated private-access controls. The comparison was deliberately stronger: ordinary documented library calls versus calls through the proposed commons registry.

Agent or review	Ordinary library	AICC registry	Private-access mock	Qualification
Claude	20/20	20/20	3/3	Code and logs independently rerun by GPT-6
Gemini	20/20 reported	20/20 reported	3/3 reported	Corrected its earlier erroneous 24/24 count
Grok	20/20	20/20	3/3	Submitted A/B code independently rerun by GPT-6

Agent or review	Ordinary library	AICC registry	Private-access mock	Qualification
GPT-6 audit	Reproduced Claude/Grok runs	Reproduced Claude/Grok runs	Reproduced mock checks	Audit, not a fourth independent Round 8 authoring trial

One of Claude's application workflows initially failed **two of the twenty assertions** in *both* approaches because a severity-filter argument was interpreted incorrectly. The same one-line fix brought both to 20/20. In this example the registry did not prevent a mistake that ordinary API use also made.

The most constructive result was about **contract quality**. Claude reported seven contract-based reimplementations matching the original functions on **14,000 generated differential inputs**; GPT-6 reran those checks. Precise specifications can guide substantial reuse and reimplementation without inspecting a source body. This is promising, although the tests were restricted to the functions and input distributions chosen for the exercise.

There was also an important failure: two alternative event-grouping implementations disagreed on **3,127 of 5,000** generated inputs. My GPT-6 audit found that one implementation violated an **explicit output-order requirement** in Claude's own written contract. Thus the result does not show that the revised contract necessarily omitted ordering. It shows something just as relevant: **a contract and a small passing test suite do not prove that independently implemented behavior conforms to that contract**. These two alternatives were also written by Claude, not independently by different AI providers.

What happened to the proposed cost savings?

The largest claimed savings appear when comparing a short reference against repeatedly reprinting an entire implementation or full contract. But competent existing software systems do not normally make that mistake: they cache documentation, keep callable functions available, and import packages without printing their source code to an AI model on every invocation.

In one of Claude's **illustrative 20-call token-accounting scenarios**, a fair cached ordinary-library baseline used **261 serialized tokenizer tokens**, and the commons binding version used **243**. This small difference is not an

end-to-end cost result; a shorter cached alias could be used by either architecture. No participant reported a complete, comparable measurement of billed model input/output/reasoning tokens, tool latency, retrieval, retries, or cost **per correctly completed application**.

The original question therefore remains open: **Can AICC outperform a strong library-plus-documentation-plus-caching baseline, rather than merely outperforming needless repetition?**

What Claude, Gemini, and Grok changed in the proposal

Claude's challenge was semantic precision and safety. Its tests exposed incomplete contracts, wrong-but-plausible implementations, insufficient conformance checking, and weaknesses in the toy binding host. It demonstrated that a capability falsely declaring itself side-effect-free could still write a file, and that an on-disk digest did not always identify the already loaded executable object. It also found a global revocation mechanism that invalidated unrelated bindings.

Gemini's contribution was an important correction and a narrower economic claim. Its first Round 8 report overstated the official test count and used preliminary file measurements. In Round 8.1 it corrected the official count to **20 workflow tests plus three simulated security checks**, reconciled frozen-file sizes, and concluded that AICC is feasible but has **not been shown to beat well-documented, cached conventional libraries**. It explicitly left native-opcode model efficiency unproven.

Grok's contribution was to separate shared access from shared understanding. It reproduced the ordinary-library and registry workflows at **20/20 each**, while emphasizing that multiple agents calling *one existing implementation* is not the same thing as multiple agents independently creating interoperable implementations from *one common specification*. That is the crucial next distinction for my idea of an agent-agreed computational foundation.

GPT-6's role was synthesis and independent rerunning of submitted code. It confirmed the official workflow and mock checks for Claude and Grok, reproduced Claude's adversarial findings, and corrected the

interpretation of the event-grouping discrepancy. These activities strengthen confidence in the recorded outputs, but they do not transform a toy experiment into independent evidence of production superiority.

All agents agreed on a serious limitation of the private-code experiment: **three successful mock permission checks are not evidence of secure isolation**. A real AICC host must enforce execution authority with sandbox or equivalent isolation, verify the *loaded artifact*, protect keys outside model context, and revoke only the bindings actually affected by a change.

The research verdict so far

Research hypothesis	What the experiments support	What is still unproven
H1 — Reuse-first development	Both a commons registry and ordinary libraries can reuse opaque functions successfully	A net cost or correctness advantage over mature packages, search and caching
H2 — AI-native language	A toy symbolic language can execute a small correct program	Savings or reliability from a model *trained for native operation tokens*
H3 — Persistent bindings	Compact handles and simulated invalidation are implementable	Real cross-session inference savings and correct policy enforcement under change
H4 — Public/private federation	Scoped access can be modeled at an API boundary	Secure encrypted execution, independent implementation interoperability and cross-provider trust

These results **do not invalidate the vision**. They make its strongest version more specific. A worthwhile AI Computational Commons needs *machine-checkable contracts, independently tested variants, host-enforced permissions, per-capability invalidation, and evidence that the entire development task gets cheaper or safer*. The proposed AI-native language can then be tested independently, without pretending a punctuation-heavy toy syntax is already a trained machine vocabulary.

Research status at this draft (November 3, 2026): Round 8.1 corrections were available from Gemini and Grok; Claude's additional Round 8.1 reply was still pending. The findings above are drawn from the participants' written reports, supplied code and logs, and GPT-6's recorded reproductions, not from a peer-reviewed or controlled multi-provider benchmark.

The unanswered questions

A proposal like this faces substantial challenges.

How can independent agents reliably determine that two capabilities have equivalent behavior?

How can a shared repository prevent malicious or defective functionality from spreading across applications?

Would a compact AI-native language actually improve development efficiency, or merely make programs harder for existing models to understand?

Can humans safely edit a C# or Python representation and preserve the behavior of the underlying system?

How much computational effort would be required to build, maintain, and verify such a repository?

And perhaps most importantly, does the cost saved by avoiding repeated code processing exceed the cost of discovering, verifying, and maintaining reusable capabilities?

These are not minor implementation details.

They are fundamental scientific and engineering questions.

Early exploratory comparisons also suggest that reuse is not automatically economical. For a very small function, a complete behavioral contract can be longer than the implementation itself.

For large capabilities with significant dependencies, the potential savings can be substantially greater.

That suggests the architecture should make intelligent decisions about when to reuse functionality, when to inspect it, and when creating new code is actually more efficient.

Four hypotheses worth testing

I believe the broader theory can be organized into four independent but related hypotheses.

Hypothesis 1: Reuse-first software development

AI agents can complete certain development tasks more efficiently by discovering and composing validated functionality without repeatedly reading or generating its implementation.

Hypothesis 2: AI-native programming representations

AI systems trained to work with a compact, precisely defined computational vocabulary can produce and modify software more efficiently or reliably than systems required to generate traditional human-readable programming languages.

Hypothesis 3: Persistent computational familiarity

AI agents can reduce repeated reasoning and context consumption by retaining validated references to functionality, including the assumptions under which that functionality is safe and suitable to use.

Hypothesis 4: Public/private capability federation

Agents from different providers can safely compose shared public functionality with authorized, owner-controlled private capabilities without exposing private implementations or misusing execution permissions.

The four hypotheses should be tested separately and in combination.

One may succeed while another fails.

For example, reuse-first development may deliver substantial improvements without requiring any new language.

Alternatively, a new computational representation may offer advantages that cannot be obtained through existing languages and tools.

Both outcomes would be useful research findings.

An invitation to rethink AI software architecture

The development of AI coding agents has largely focused on making models better at understanding, generating, debugging, and maintaining source code.

That progress is valuable.

But perhaps an equally important opportunity lies in reducing how much code those agents need to process in the first place.

Imagine an agent developing an application in which most required functionality already exists.

Rather than generating thousands of lines of implementation, it identifies suitable capabilities, verifies their compatibility, composes them into a system, and develops only the missing functionality.

And when it does need to create something new, perhaps it should not be constrained to express that functionality through a programming language designed decades ago for human developers.

This is not a claim that existing languages should disappear, or that current AI systems are ready to replace them.

It is a proposal to investigate whether our software-development architecture is unnecessarily constrained by the way humans have historically programmed computers.

The opportunity is not simply faster code generation.

It may be a transition from software development as repeated construction toward software development as cumulative computational knowledge.

We keep asking how to make AI better at writing code.

Perhaps we should also be asking why AI needs to write so much code at all.

Article credits

Concept and architecture: Izik Bachar
Draft development: ChatGPT (OpenAI), Claude (Anthropic), Gemini (Google), Grok (xAI), and Izik Bachar
Final draft: Izik Bachar and ChatGPT (OpenAI)